

DIGITAL IMAGE PROCESSING

MINI PROJECT REPORT

Coin Detection and Counting Using Image Segmentation Techniques

Submitted by
Adithyan PM

BCA – Digital Image Processing Mini Project

24UBC205

Abstract

This mini project presents a coin detection and counting system based on digital image processing and image segmentation techniques. The objective is to automatically identify coin-shaped regions in an image and estimate the total number of coins without manually counting them. The implemented pipeline uses grayscale conversion, median filtering, histogram analysis, thresholding, Otsu's thresholding, morphological processing, contour detection, and area-based filtering. Three test images with different coin arrangements and backgrounds were evaluated. The system correctly detected all 15 coins in the first image, detected 58 against 57 actual coins in the second, and detected 9 against 6 actual coins in the overlapping-coin test image. The average counting agreement accuracy across the three images was 82.75%.

1. Introduction

Digital Image Processing (DIP) provides techniques for improving, transforming, and analysing digital images. Image segmentation is an important part of DIP because it separates meaningful objects or regions from the background. Once objects are segmented, their boundaries and geometric properties can be analysed using techniques such as contour detection.

Coin counting is a practical visual analysis problem. Manual counting can be inconvenient when many coins are present or when coins are arranged in a complex way. This project investigates a simple image-processing approach in Python for detecting separated coin regions and counting them automatically.

2. Problem Statement

Develop an image-processing application that detects and counts coins from digital images using segmentation and contour analysis. The system should preprocess the input, separate coin regions from the background, remove small unwanted regions, identify valid coin contours, and report the detected coin count.

3. Objectives

- Understand a real-world application of image segmentation.
- Preprocess coin images using grayscale conversion and median filtering.
- Compare fixed/global thresholding with Otsu's automatic thresholding.
- Improve binary segmentation using morphological opening and closing.
- Detect coin boundaries using external contours.
- Remove small unwanted regions using contour-area filtering.
- Count detected coins and compare the result with manually determined counts.
- Identify limitations and propose improvements for overlapping and difficult coin images.

4. Dataset and Image Description

A small test dataset of three coin images was used. The images were collected from public web image sources and represent different visual conditions. The dataset was kept small so that processing stages and results could be inspected manually during the mini-project study.

Image	Description	Purpose
dip3.jpeg	Coins on a relatively clear background.	Basic separated-coin test.

international-coins.jpg	Many coins with varied appearances on a light background.	Normal multi-coin test.
famous-heads-coin-collection.jpg	Coins with significant touching/overlap.	Challenging/failure-case test.

5. Tools and Technologies

- Python – implementation language.
- OpenCV – image reading, color conversion, filtering, thresholding, morphology, and contours.
- NumPy – numerical arrays and kernel creation.
- Matplotlib – histogram and result visualisation.
- Pandas – storing and comparing experimental results.
- Google Colab – interactive development and execution environment.
- GitHub – source-code, image, screenshot, report, and project-file hosting.

6. Methodology

Input Image → Grayscale → Median Filtering → Histogram → Thresholding/Otsu → Morphological Processing → Contour Detection → Area Filtering → Counting → Evaluation

6.1 Image Acquisition

The selected image is loaded using OpenCV. The original colour image is retained for final visualisation.

6.2 Grayscale Conversion

The BGR input is converted to a single-channel grayscale image, simplifying intensity-based segmentation.

6.3 Median Filtering

A 5×5 median filter is applied to reduce small noise while preserving useful boundaries.

6.4 Histogram Analysis

The grayscale intensity histogram is examined to understand the distribution of foreground and background intensities.

6.5 Global Thresholding

A fixed threshold is tested to convert the grayscale image into a binary representation.

6.6 Otsu Thresholding

Otsu's method automatically selects a threshold from the intensity distribution and is used as the main thresholding approach.

6.7 Morphological Processing

A 5×5 kernel is used with morphological opening followed by closing to remove small regions and improve connected foreground regions.

6.8 Contour Detection

External contours are extracted from the processed binary image using OpenCV.

6.9 Area-Based Filtering

Contours smaller than an experimentally selected minimum area are rejected as noise or irrelevant regions.

6.10 Coin Counting

The number of remaining valid contours is treated as the detected coin count. Bounding boxes and contour outlines are drawn for visual verification.

7. Implementation

The implementation was developed incrementally in Google Colab. The screenshots in this report document the actual implementation stages, including image loading, grayscale conversion, median filtering, histogram analysis, thresholding, Otsu thresholding, morphology, contour detection, final detections, and accuracy comparison.

- `cv2.imread()` – reads the input image.
- `cv2.cvtColor()` – converts the image to grayscale.
- `cv2.medianBlur()` – performs median filtering.
- `cv2.threshold()` with Otsu – obtains an automatic threshold.
- `cv2.morphologyEx()` – performs opening and closing.
- `cv2.findContours()` – extracts external object contours.
- `cv2.contourArea()` – measures contour area for filtering.
- `cv2.boundingRect()` – obtains a bounding box for each detected object.

8. Implementation Screenshots

The following screenshots document the main stages of the project implementation in Google Colab.

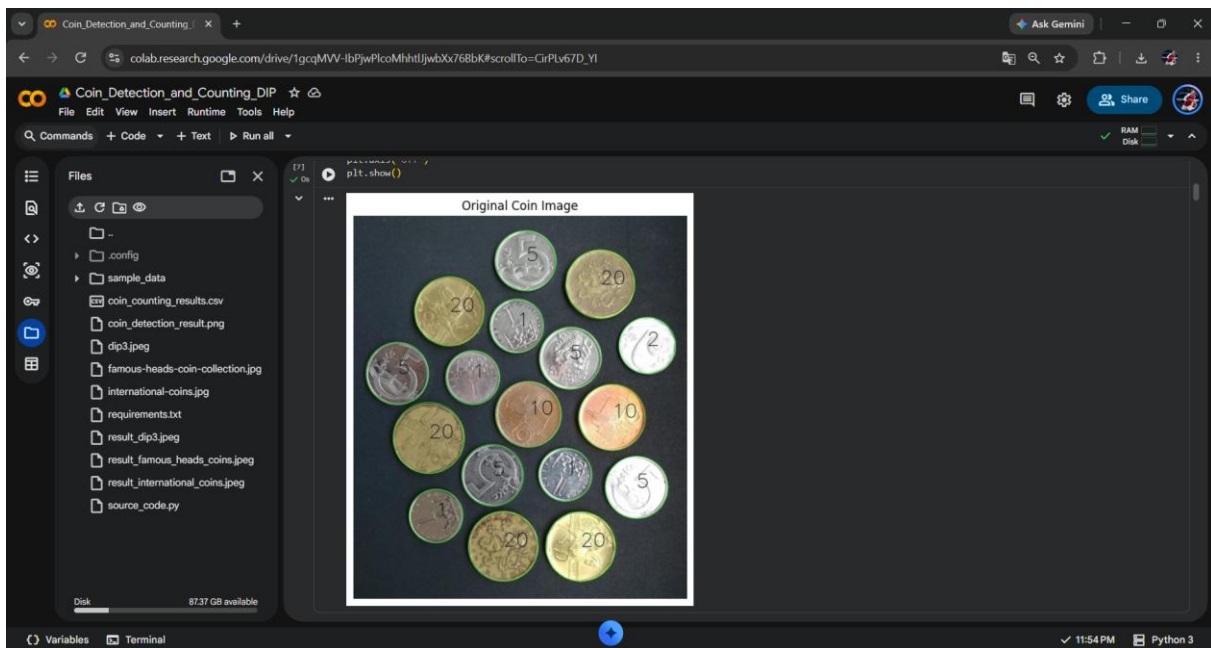


Figure 1: Original coin image displayed in Google Colab.

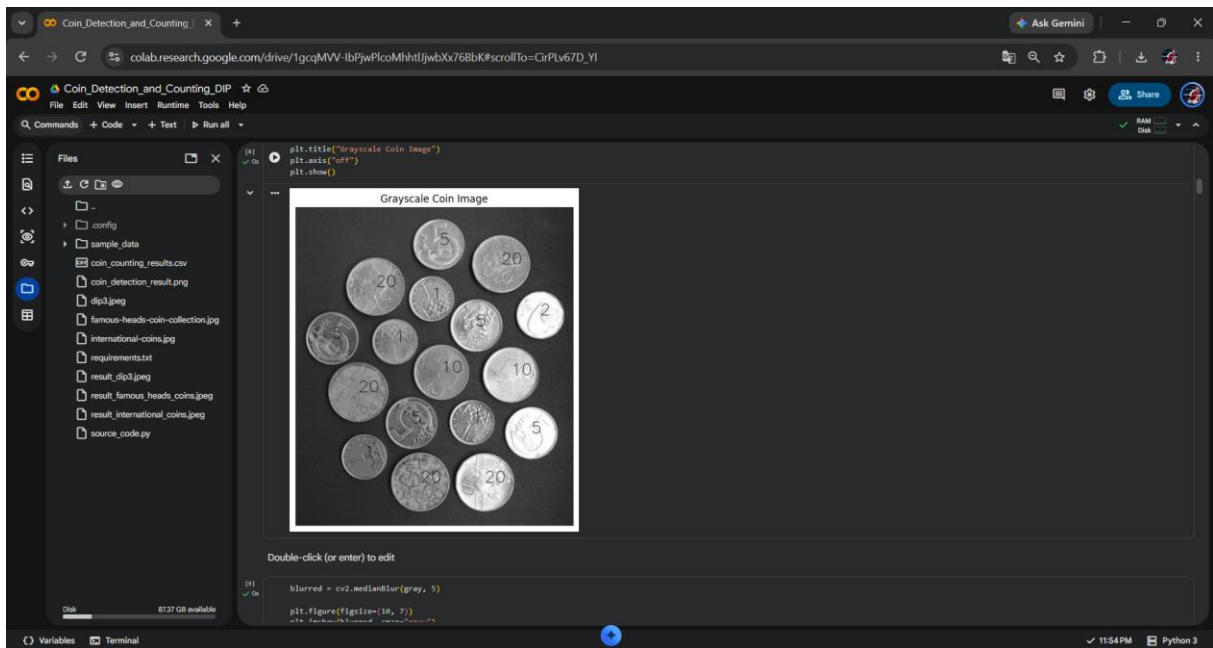


Figure 2: Grayscale coin image and preprocessing stage.

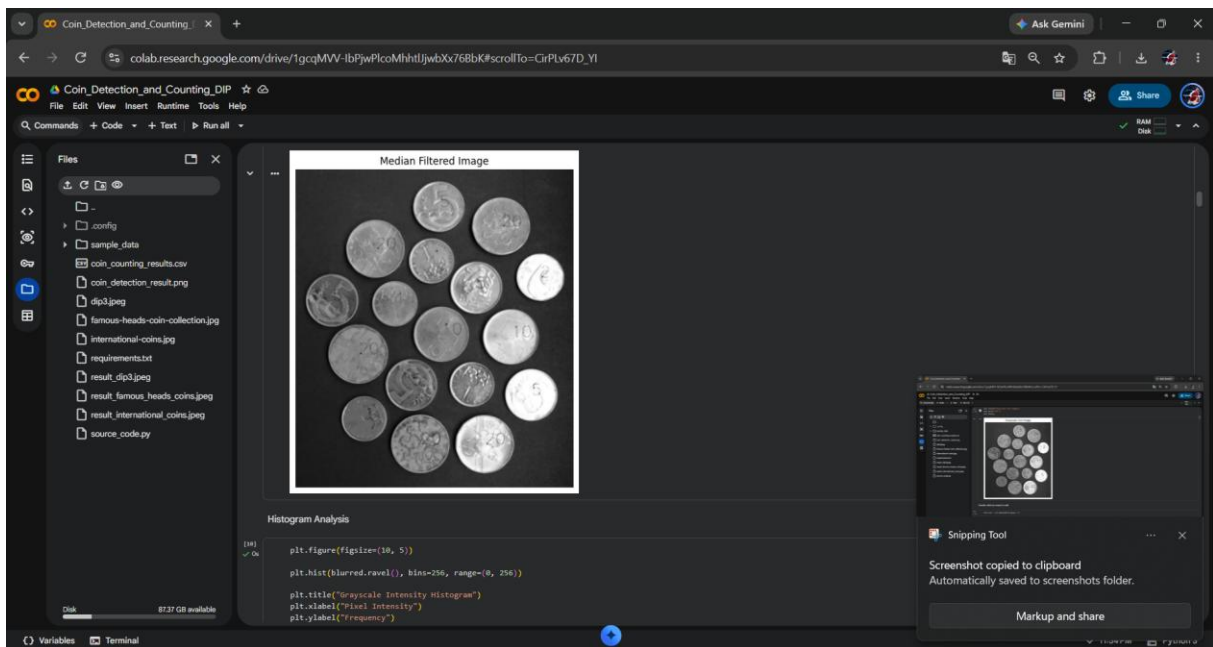


Figure 3: Median filtered image and histogram analysis.

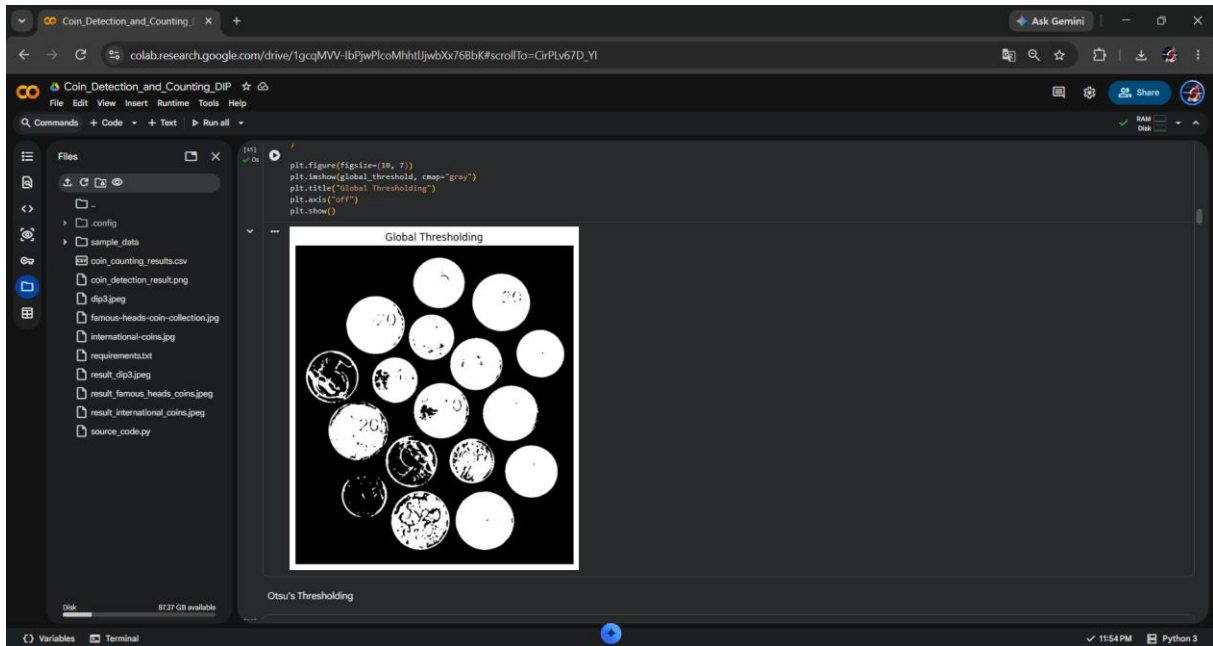


Figure 4: Global thresholding result.

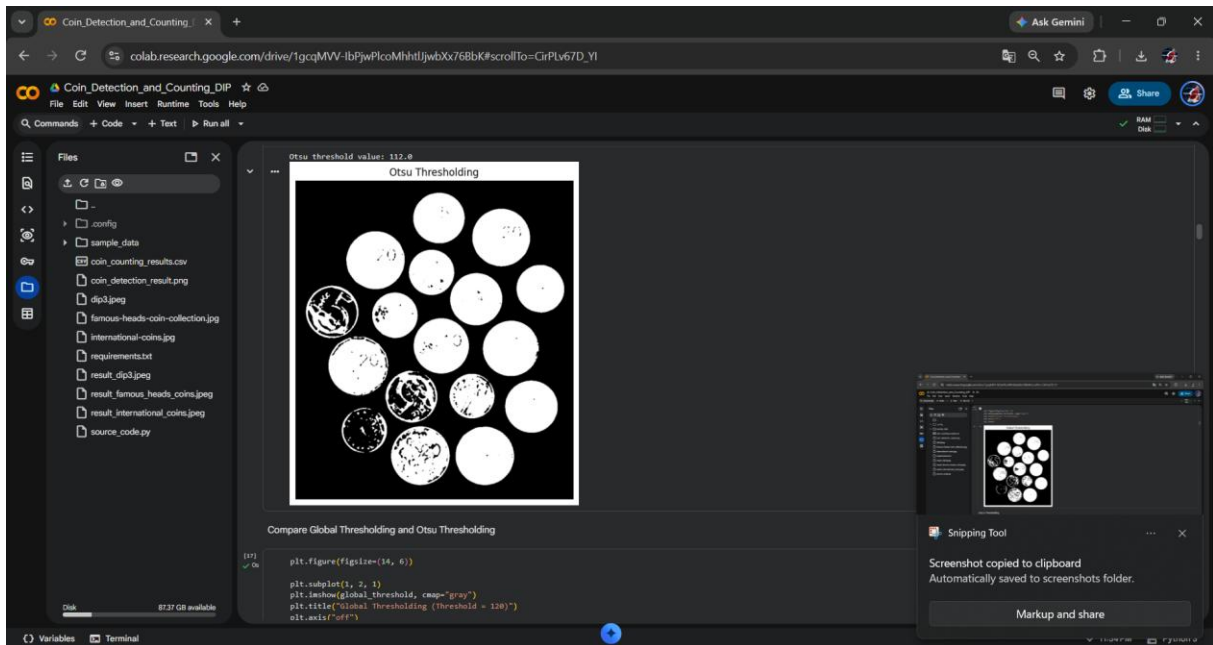


Figure 5: Otsu thresholding result; Otsu threshold value shown as 112.0.

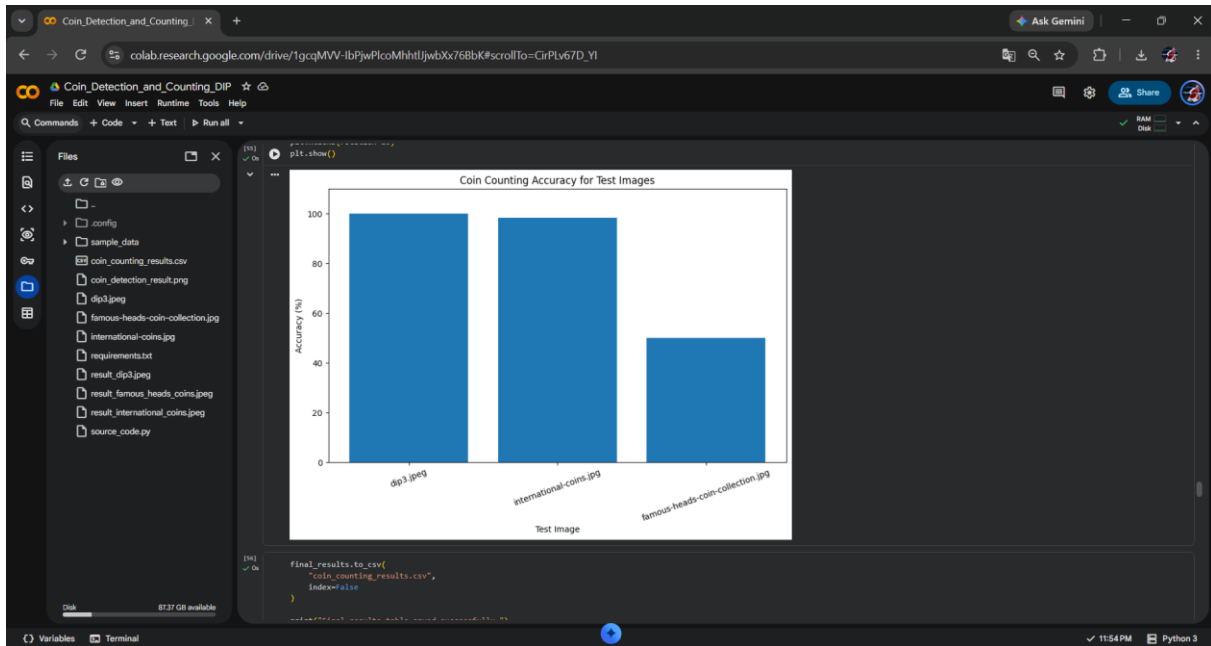


Figure 8: Coin counting accuracy comparison for the three test images.

9. Results

The system was tested on three images. Actual counts were obtained by manual inspection, while detected counts were produced by the implemented contour-based pipeline.

Image	Actual	Detected	Error	Accuracy (%)
dip3.jpeg	15	15	0	100.00
international-coins.jpeg	57	58	1	98.25
famous-heads-coin-collection.jpeg	6	9	3	50.00

Average counting agreement accuracy: 82.75%.

Total absolute counting error across the three test images: 4 coins.

10. Evaluation and Interpretation

A simple counting agreement accuracy was used:

$$\text{Accuracy} = (1 - |\text{Actual} - \text{Detected}| / \text{Actual}) \times 100$$

This metric describes how closely the detected count agrees with the manually determined count. It is not the same as object-detection precision, recall, or segmentation IoU because individual coin-level ground-truth masks and detections were not prepared.

For dip3.jpeg, the method detected all 15 coins, giving 100% count agreement. For international-coins.jpeg, one extra region was counted, giving 98.25%. For famous-heads-coin-collection.jpeg, overlapping/touching coins caused multiple incorrect regions to be produced, resulting in 50% count agreement.

11. Comparison of Thresholding Approaches

Approach	Description	Observation
Global thresholding	Uses a fixed threshold selected manually.	Useful for a simple fixed segmentation condition, but performance depends on the selected value.
Otsu thresholding	Automatically selects a threshold from the intensity distribution.	Used as the main approach because it reduces manual threshold selection.

12. Limitations

- The method works best when coins are clearly separated from each other.
- Overlapping or touching coins may be detected as a single region or may produce inaccurate counts.
- The threshold value and minimum contour area can affect the detection result.
- Lighting, shadows, reflections, and background can affect segmentation.
- Coins with very similar intensity to the background may be difficult to separate.
- The current method mainly uses contour area and does not identify denomination or monetary value.
- Only three test images were evaluated, so the measured average should not be treated as a general performance estimate.

13. Future Scope

- Watershed segmentation can be used to separate touching and overlapping coins.
- Adaptive thresholding can be tested for images with uneven illumination.
- Hough Circle Transform can be investigated for circular coin detection.
- Coin denomination recognition can be added using image classification techniques.
- A larger and more diverse dataset can be used for more reliable evaluation.
- The system can be developed into a real-time coin counting application using a camera.

14. Conclusion

This project implemented a coin detection and counting system using digital image processing techniques. The process included grayscale conversion, median filtering, histogram analysis, thresholding, Otsu's method, morphological processing, contour detection, and area-based filtering.

The system was tested on multiple coin images containing different arrangements and backgrounds. The results showed that the method can effectively detect coins when they are sufficiently separated. The challenging overlapping-coin image also demonstrated the limitations of basic contour-based segmentation.

Overall, the project demonstrates how image segmentation and contour analysis can be applied to a practical visual problem such as automatic coin counting.

15. Project Files and GitHub Structure

- source_code.py – clean Python implementation.
- requirements.txt – required Python packages.
- coin_counting_results.csv – experimental results table.
- images/ – original test images and generated detection results.

- screenshots/ – screenshots documenting the implementation stages.
- report/ – final project report.
- README.md – project overview and usage information.

16. References

- OpenCV documentation – image processing, thresholding, morphology, and contour analysis.
- NumPy documentation – numerical arrays and matrix operations.
- Matplotlib documentation – plotting and histogram visualisation.
- Pandas documentation – tabular data handling and result comparison.
- Course-provided Digital Image Processing Mini Project Guidelines.
- Public web image sources used for the selected coin test images.